notebook

March 23, 2026

0.1 1. Getting familiar with optimization test problems

[1]:
```
import numpy as np
from abc import ABC, abstractmethod

class Problem(ABC):
    @abstractmethod
    def __call__(self, x: np.ndarray) -> float:
        """Compute the function value at point x."""
        raise NotImplementedError

    @abstractmethod
    def grad(self, x: np.ndarray) -> np.ndarray:
        """Compute the gradient at point x."""
        raise NotImplementedError

class Sphere(Problem):
    def __call__(self, x: np.ndarray) -> float:
        return np.sum(x**2)

    def grad(self, x: np.ndarray) -> np.ndarray:
        return 2 * x

class Rosenbrock(Problem):
    def __call__(self, x: np.ndarray) -> float:
        return np.sum(100.0 * (x[1:] - x[:-1]**2)**2 + (1 - x[:-1])**2)

    def grad(self, x: np.ndarray) -> np.ndarray:
        grad = np.zeros_like(x)
        n = x.size
        grad[0] = -400 * x[0] * (x[1] - x[0]**2) - 2 * (1 - x[0])
        for i in range(1, n - 1):
            grad[i] = 200 * (x[i] - x[i-1]**2) - 400 * x[i] * (x[i+1] -␣
 ↪x[i]**2) - 2 * (1 - x[i])
        grad[-1] = 200 * (x[-1] - x[-2]**2)
        return grad

class Rastrigin(Problem):
```

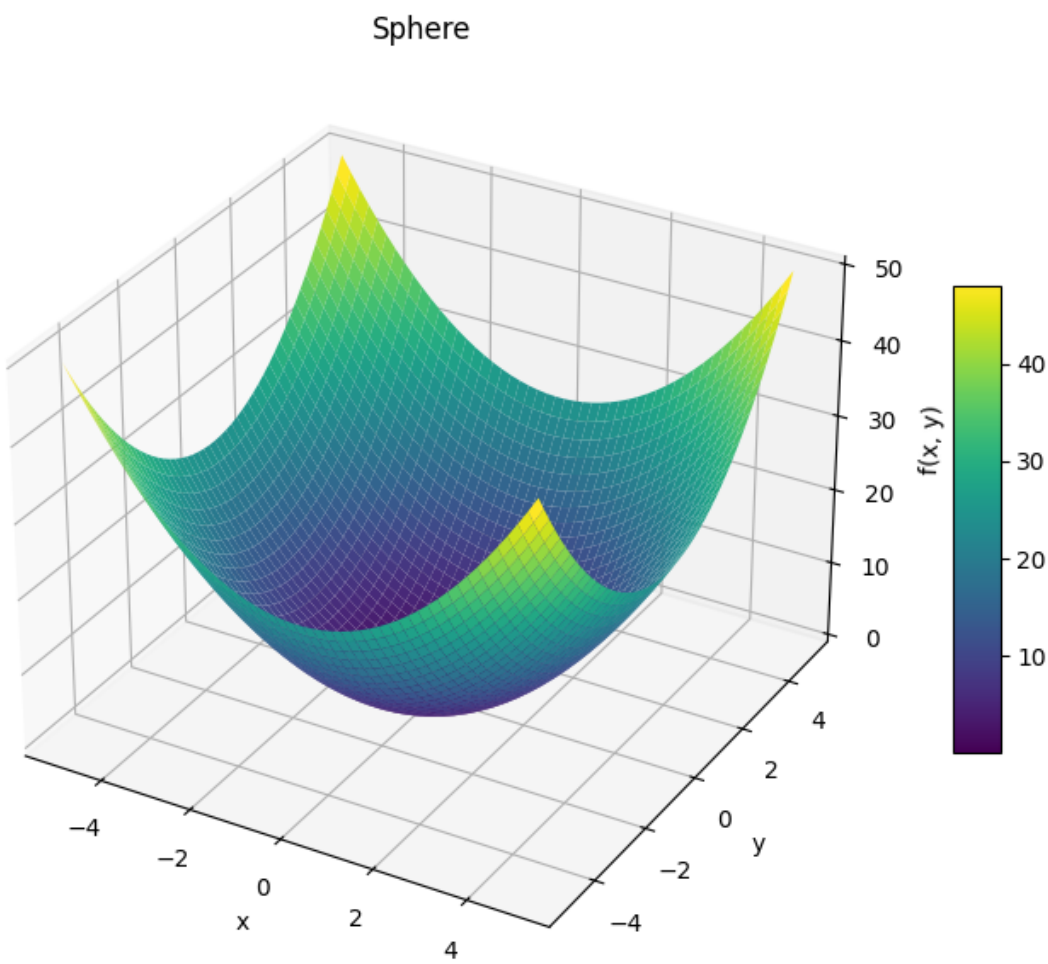
```
    def __call__(self, x: np.ndarray) -> float:
        A = 10
        n = x.size
        return A * n + np.sum(x**2 - A * np.cos(2 * np.pi * x))

    def grad(self, x: np.ndarray) -> np.ndarray:
        A = 10
        return 2 * x + 2 * np.pi * A * np.sin(2 * np.pi * x)
```

[2]:
```
import matplotlib.pyplot as plt
import plotly.graph_objects as go

def prepare_mesh_grid(
    problem: Problem,
    bounds: tuple[float, float] = (-5.0, 5.0),
    grid_size: int = 50,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    x_vals = np.linspace(bounds[0], bounds[1], grid_size)
    y_vals = np.linspace(bounds[0], bounds[1], grid_size)
    X, Y = np.meshgrid(x_vals, y_vals)

    Z = np.zeros_like(X)
    for i in range(grid_size):
        for j in range(grid_size):
            xy = np.array([X[i, j], Y[i, j]])
            Z[i, j] = problem(xy)

    return X, Y, Z, x_vals, y_vals


def plot_3d_surface(
    problem: Problem,
    grid_size: int = 50,
):
    X, Y, Z, _, _ = prepare_mesh_grid(problem=problem, grid_size=grid_size)

    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection="3d")
    surf = ax.plot_surface(X, Y, Z, cmap="viridis", edgecolor="none")

    ax.set_title(problem.__class__.__name__)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("f(x, y)")

    fig.colorbar(surf, shrink=0.5, aspect=10)
    plt.tight_layout()
```

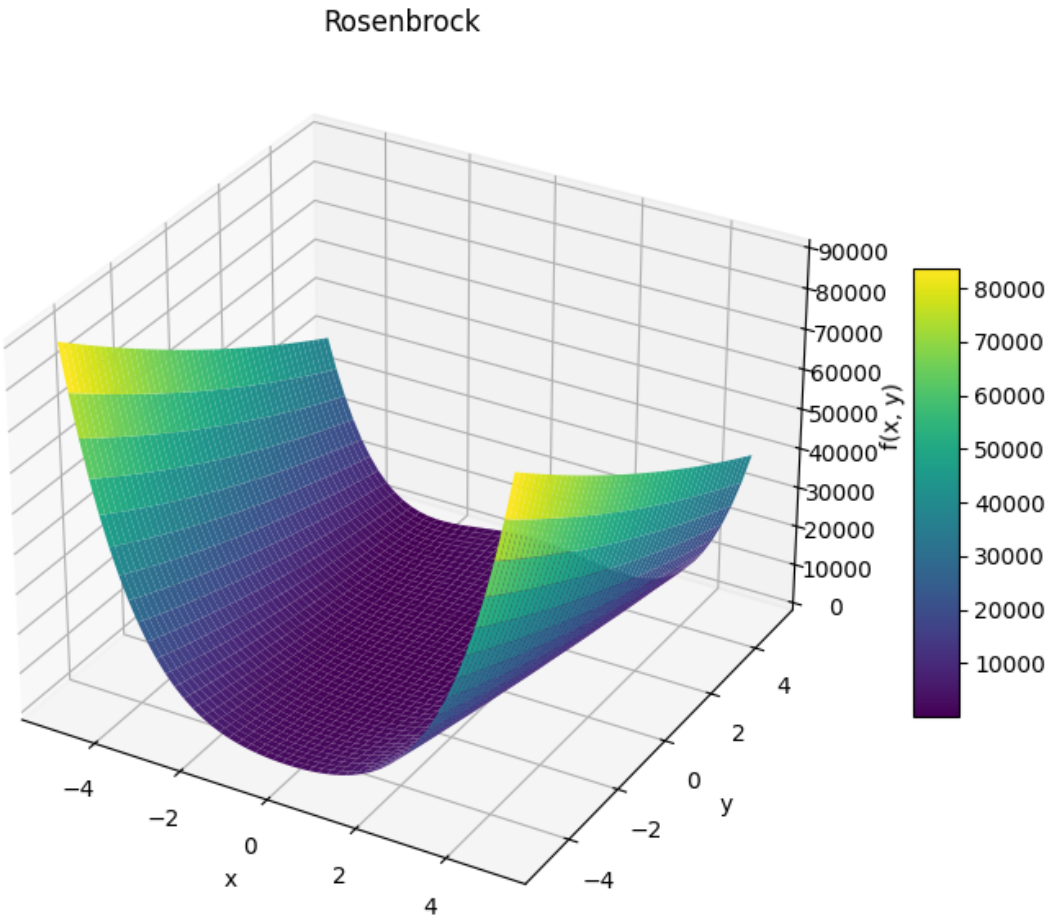
```
    plt.show()

def plot_contour_and_paths(
    problem: Problem,
    paths: list[np.ndarray],
    grid_size: int = 200,
    title: str = "",
):
    """
    Create an interactive contour plot of a 2D function and overlay multiple␣
 ↪optimization paths.

    Args:
        problem: An instance of a Problem class.
        paths: List of numpy arrays; each array is of shape (epochs, 2)␣
 ↪containing an optimization trajectory.
        title: Title for the plot.
    """
    _, _, Z, x_vals, y_vals = prepare_mesh_grid(problem, grid_size=grid_size)

    fig = go.Figure(
        data=go.Contour(
            x=x_vals,
            y=y_vals,
            z=Z,
            colorscale="Viridis",
            contours=dict(showlines=False),
            colorbar=dict(title="Function Value"),
        )
    )

    colors = ['red', 'blue', 'green', 'purple', 'orange', 'cyan', 'magenta',␣
 ↪'yellow', 'pink', 'brown']

    for idx, path in enumerate(paths):
        color_idx = idx % len(colors)
        fig.add_trace(
            go.Scatter(
                x=path[:, 0],
                y=path[:, 1],
                mode="lines+markers",
                marker=dict(size=4),
                line=dict(width=2, color=colors[color_idx]),
                name=f"Run {idx+1}",
                showlegend=True,
            )
        )
```

```
    fig.update_layout(
        title=title, xaxis_title="x", yaxis_title="y", width=800, height=700,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right",␣
↪x=1)
    )

    fig.show()
```

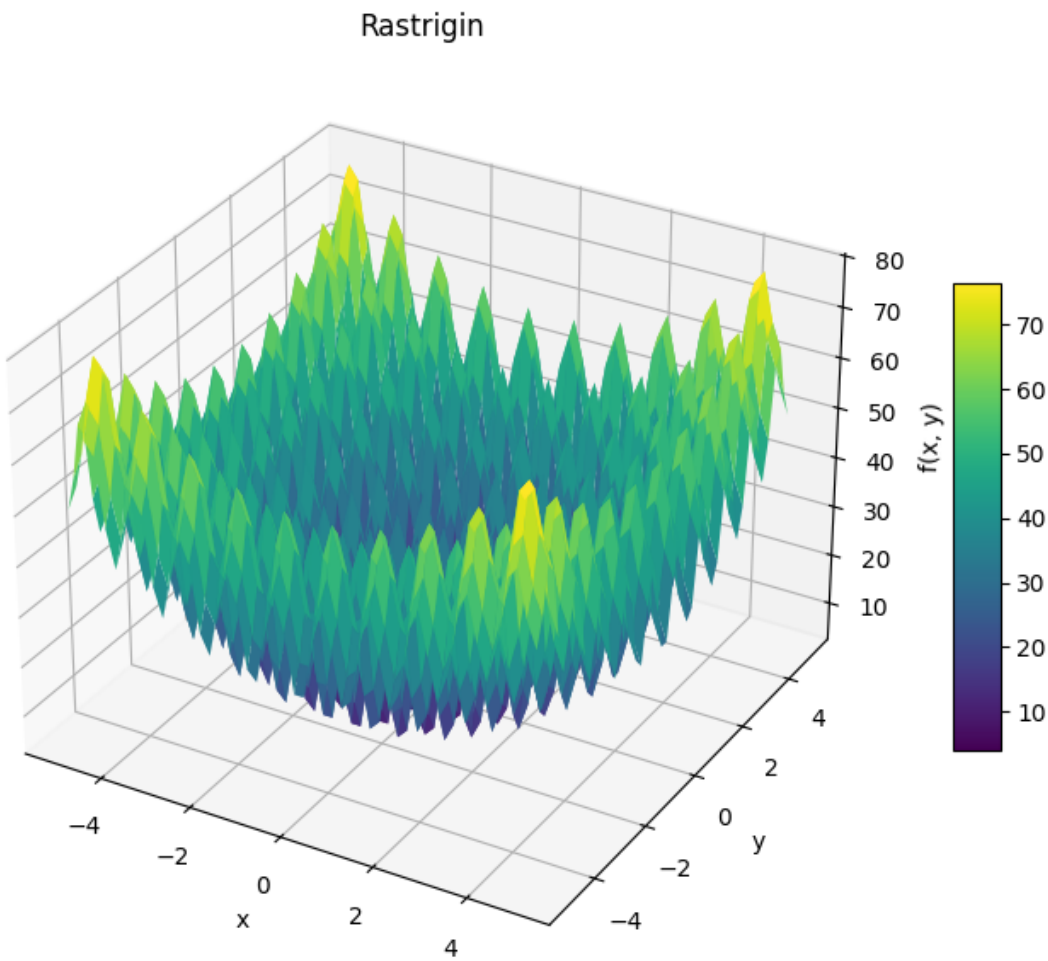
[3]:
```
sphere = Sphere()
plot_3d_surface(problem=sphere)
```



[4]:
```
rosenbrock = Rosenbrock()
plot_3d_surface(rosenbrock)
```



[5]:
```
rastrigin = Rastrigin()
plot_3d_surface(problem=rastrigin)
```



Question: Which function appears easiest to optimize? Hardest? Why?

Answer: The `Sphere` seems to be the easiest one to optimize as it has one, clear global minimum. The `Rastrigin` will be the hardest one in terms of finding global minimum as optimizers are likely to get stuck in one of the "holes" and find local minimum only (which one strongly depends on the starting point). The `Rosenbrock` problem brings issue of different type - finding the "valley" should be easy for optimizer, but navigating it and finding real global minimum might be hard because of very small gradients.

0.2 2. Momentum method

[6]:
```
def momentum(
    problem: Problem,
    initial_solution: np.ndarray,
    alpha: float,
    beta: float,
    number_of_epochs: int,
```

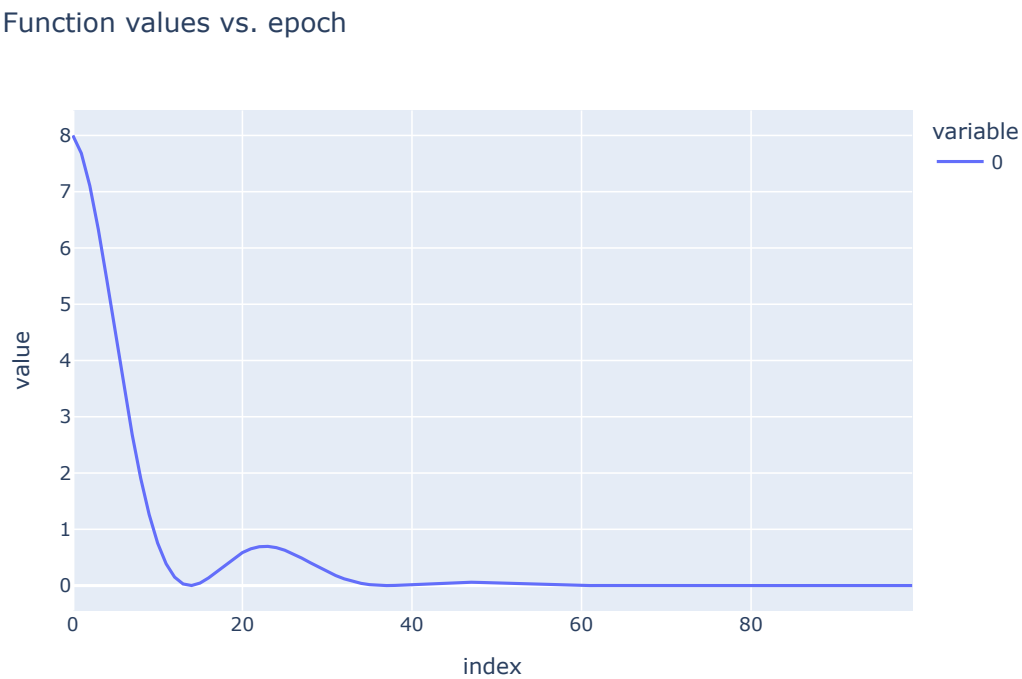
```
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    dim = initial_solution.shape[0]
    v = np.zeros_like(initial_solution)
    w = initial_solution

    positions = np.ndarray(shape=(number_of_epochs, dim))
    function_values = np.ndarray(shape=(number_of_epochs))
    velocity_norms = np.ndarray(shape=(number_of_epochs))

    positions[0] = w
    function_values[0] = problem(w)
    velocity_norms[0] = np.linalg.norm(v)

    for i in range(1, number_of_epochs):
      v = beta * v + alpha * problem.grad(w)
      w = w - v

      positions[i] = w
      function_values[i] = problem(w)
      velocity_norms[i] = np.linalg.norm(v)

    return positions, function_values, velocity_norms
```

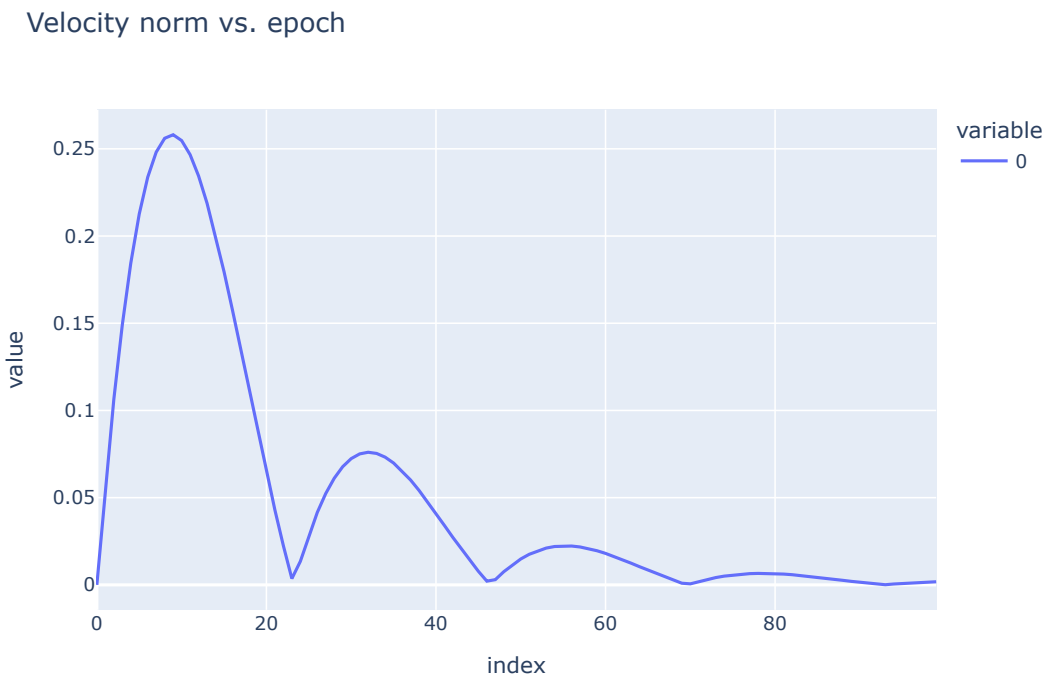
[7]:
```
sphere = Sphere()

_sphere_positions, _sphere_function_values, _sphere_velocity_norms = momentum(
    problem=sphere,
    initial_solution=np.array([2.0, 2.0]),
    alpha=0.01,
    beta=0.9,
    number_of_epochs=100
)

sphere_positions = [_sphere_positions]
sphere_function_values=[_sphere_function_values]
sphere_velocity_norms=[_sphere_velocity_norms]
```

[8]:
```
import plotly.io as pio
pio.renderers.default = "notebook+pdf"
```

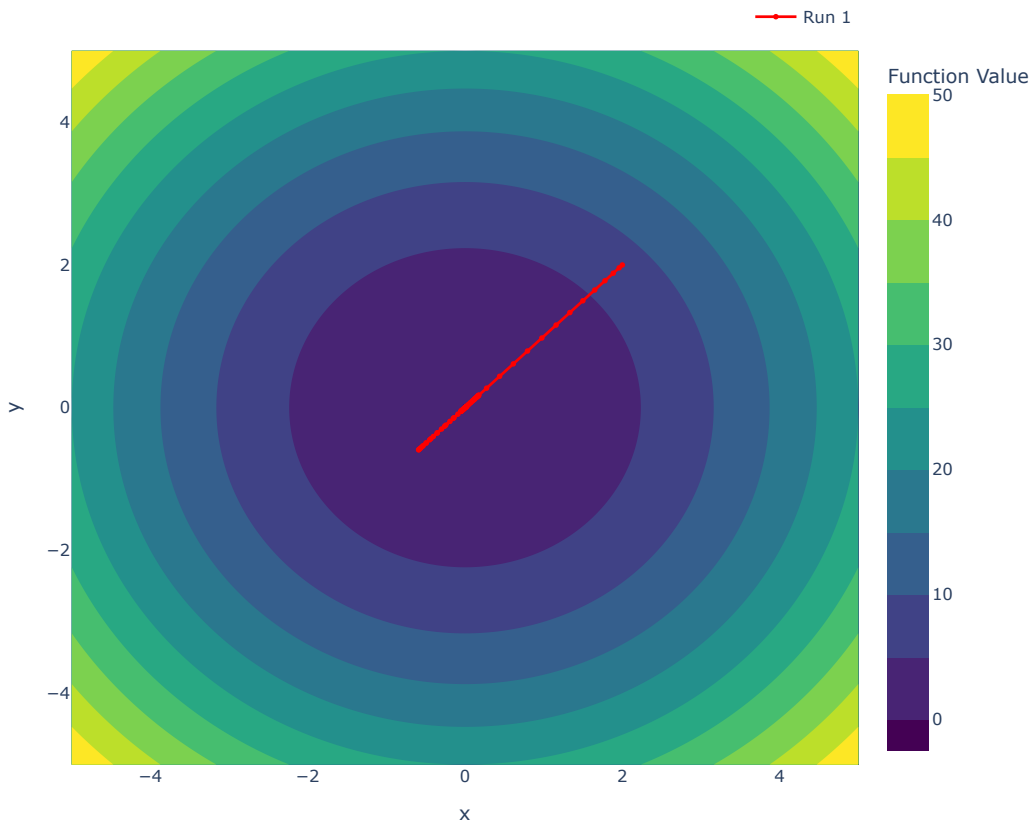
[9]:
```
import plotly.express as px

fig1 = px.line(
  sphere_function_values[0],
  title="Function values vs. epoch",
)
fig2 = px.line(
```

```
  sphere_velocity_norms[0],
  title="Velocity norm vs. epoch"
)

fig1.show()
fig2.show()
```





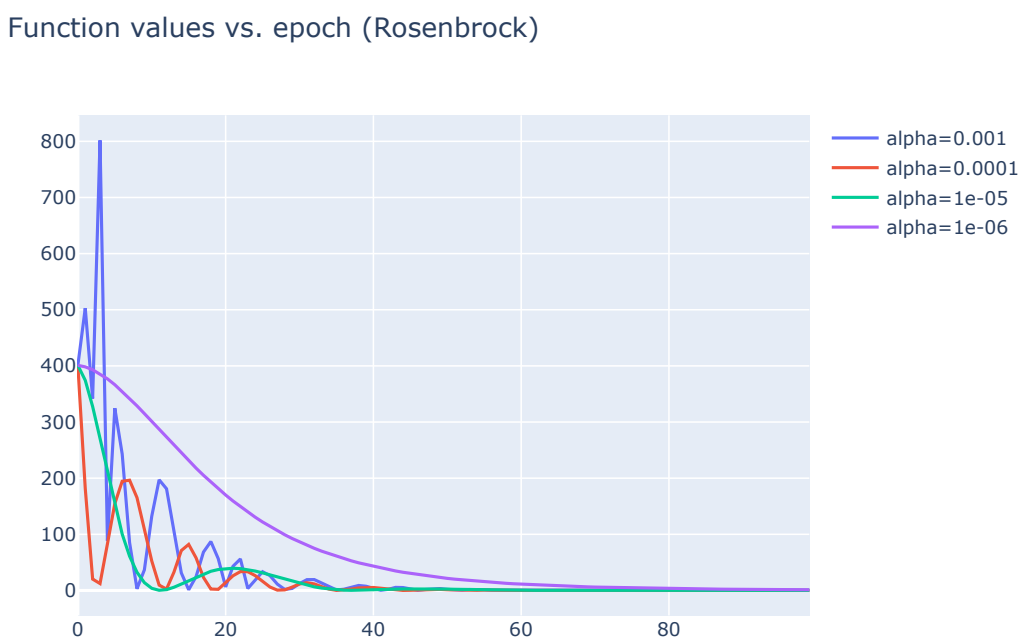
[10]:
```
plot_contour_and_paths(problem=sphere, paths=sphere_positions)
```



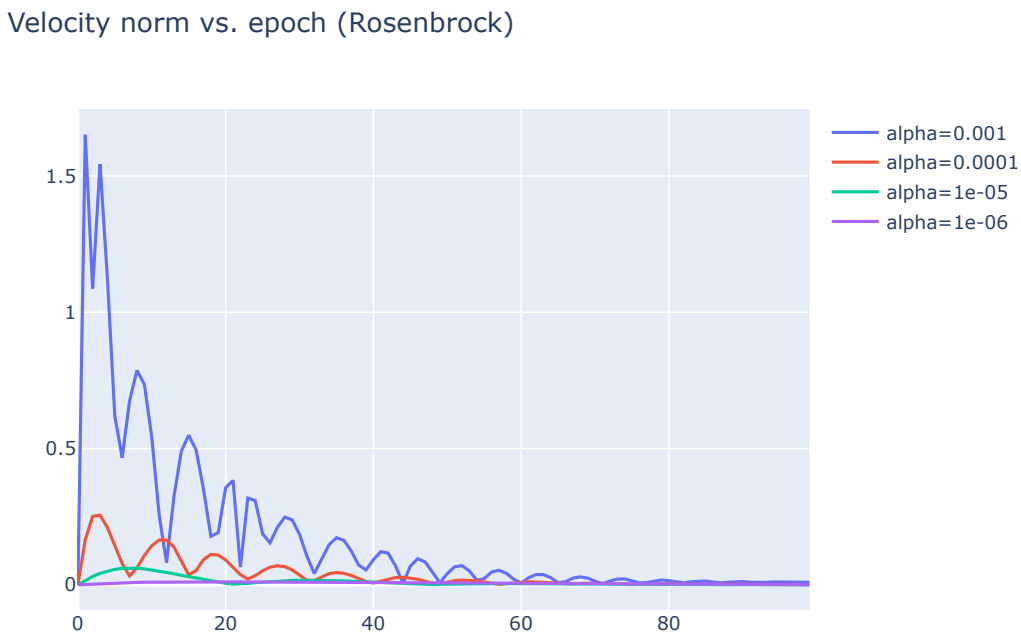
[11]:
```
alphas = [1e-3, 1e-4, 1e-5, 1e-6]

rosenbrock = Rosenbrock()
rosenbrock_positions = []
rosenbrock_function_values = []
rosenbrock_velocity_norms = []

for alpha in alphas:
  _positions, _function_values, _velocity_norms = momentum(
      problem=rosenbrock,
      initial_solution=np.array([2.0, 2.0]),
      alpha=alpha,
      beta=0.9,
      number_of_epochs=100
  )
```

```
  rosenbrock_positions.append(_positions)
  rosenbrock_function_values.append(_function_values)
  rosenbrock_velocity_norms.append(_velocity_norms)
```

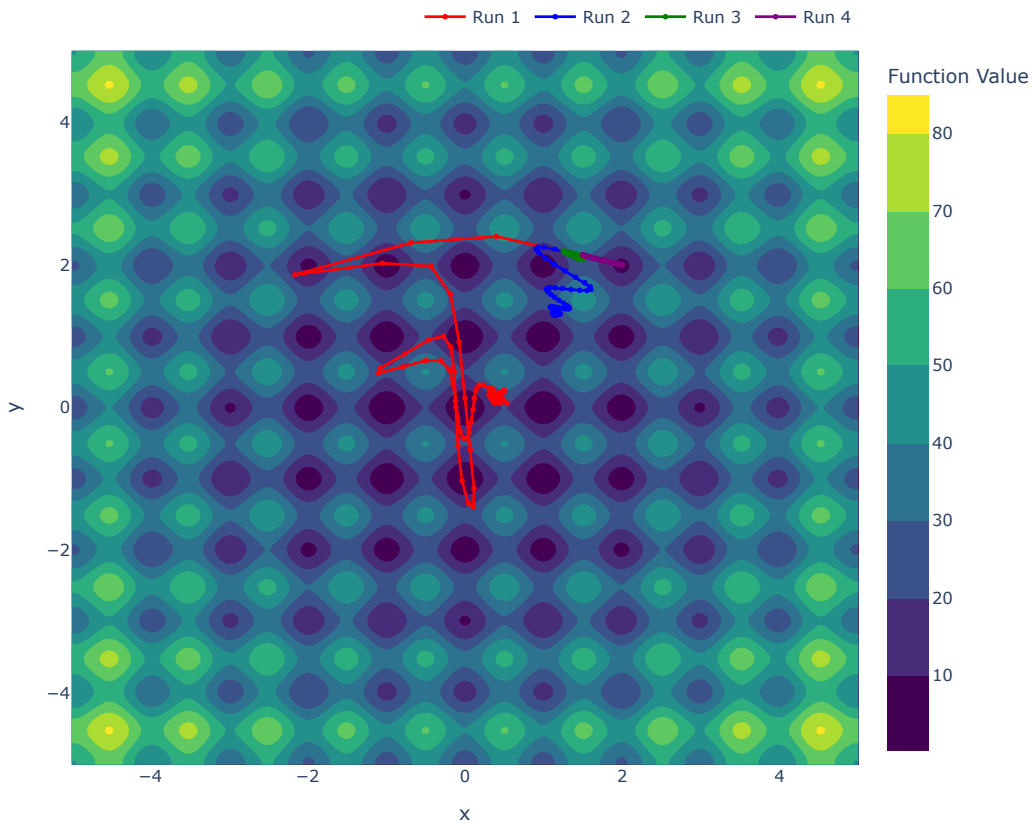
[12]:
```
import plotly.graph_objects as go

fig = go.Figure()
for alpha, function_values in zip(alphas, rosenbrock_function_values):
  fig.add_trace(go.Scatter(y=function_values, name=f"alpha={alpha}"))
fig.update_layout(
    title="Function values vs. epoch (Rosenbrock)"
)
```



[13]:
```
import plotly.graph_objects as go

fig = go.Figure()
for alpha, velocity_norms in zip(alphas, rosenbrock_velocity_norms):
  fig.add_trace(go.Scatter(y=velocity_norms, name=f"alpha={alpha}"))
fig.update_layout(
    title="Velocity norm vs. epoch (Rosenbrock)"
)
```



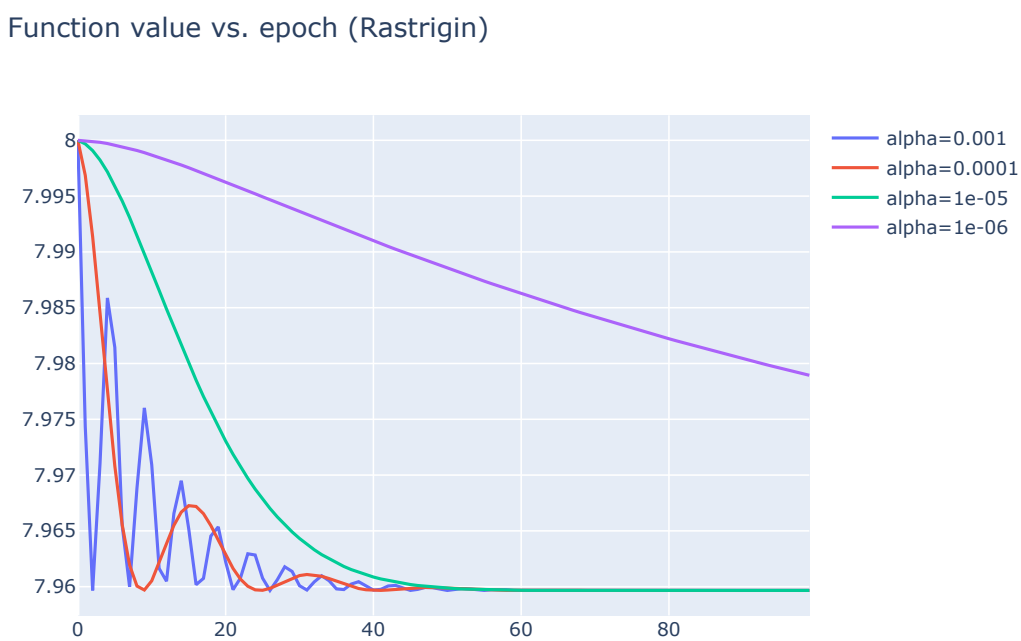
[14]:
```
plot_contour_and_paths(problem=rastrigin, paths=rosenbrock_positions)
```



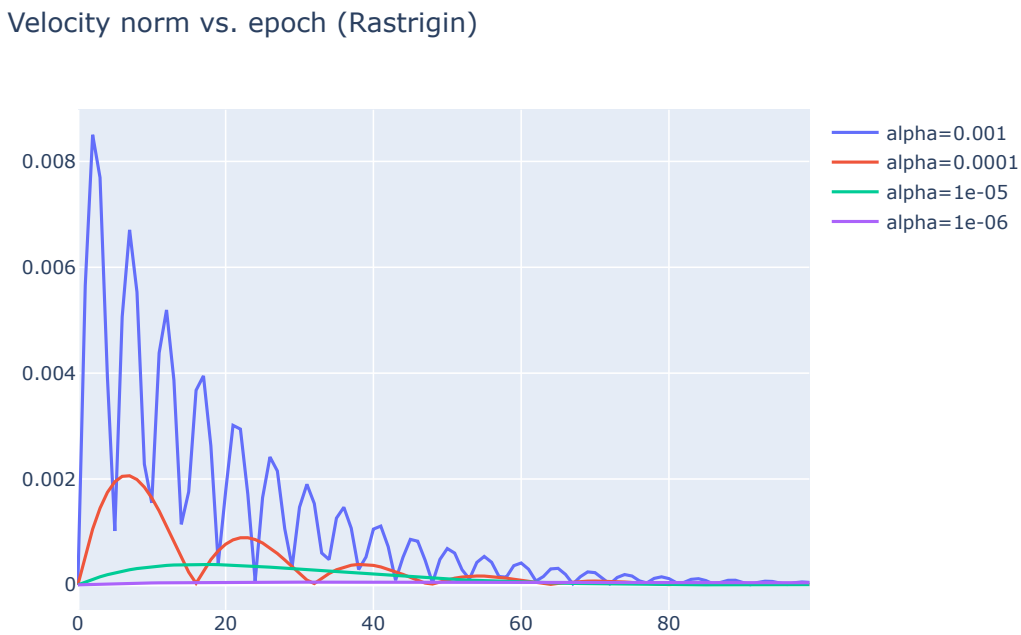
[15]:
```
alphas = [1e-3, 1e-4, 1e-5, 1e-6]

rastrigin = Rastrigin()
rastrigin_positions = []
rastrigin_function_values = []
rastrigin_velocity_norms = []

for alpha in alphas:
  _positions, _function_values, _velocity_norms = momentum(
      problem=rastrigin,
      initial_solution=np.array([2.0, 2.0]),
      alpha=alpha,
      beta=0.9,
      number_of_epochs=100
  )
```

```
  rastrigin_positions.append(_positions)
  rastrigin_function_values.append(_function_values)
  rastrigin_velocity_norms.append(_velocity_norms)
```

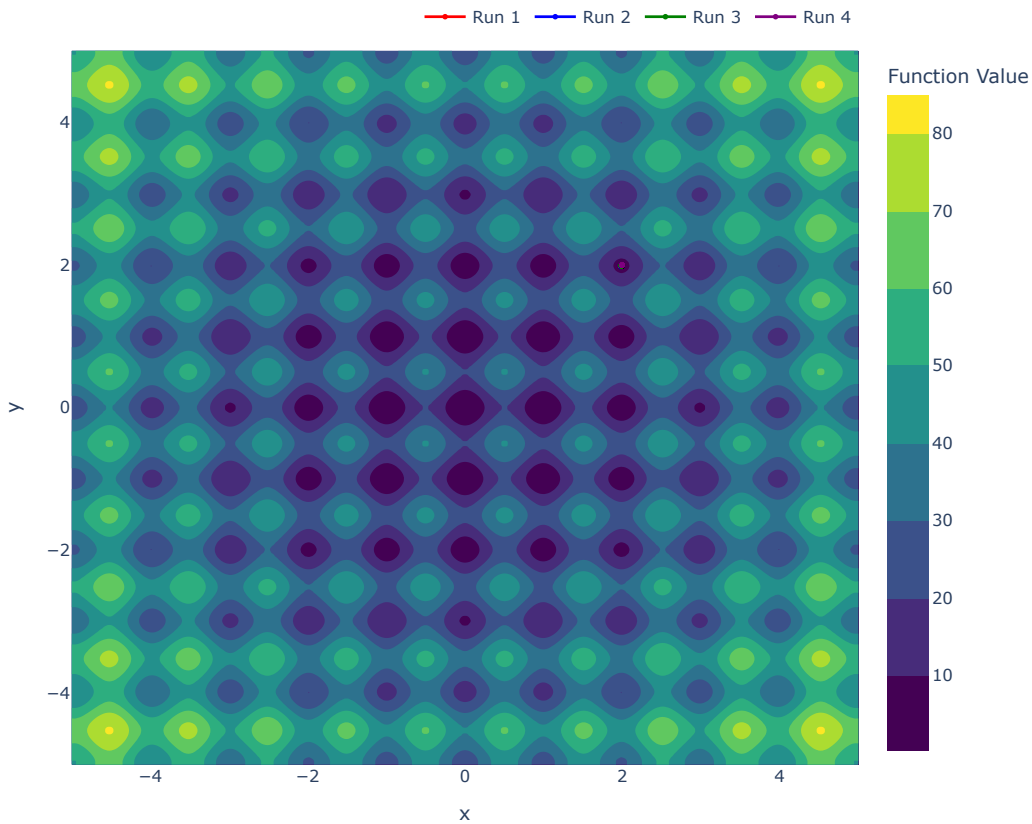
[16]:
```
import plotly.graph_objects as go

fig = go.Figure()
for alpha, function_values in zip(alphas, rastrigin_function_values):
  fig.add_trace(go.Scatter(y=function_values, name=f"alpha={alpha}"))
fig.update_layout(
    title="Function value vs. epoch (Rastrigin)"
)
```



[17]:
```
import plotly.graph_objects as go

fig = go.Figure()
for alpha, velocity_norms in zip(alphas, rastrigin_velocity_norms):
  fig.add_trace(go.Scatter(y=velocity_norms, name=f"alpha={alpha}"))
fig.update_layout(
    title="Velocity norm vs. epoch (Rastrigin)"
)
```



[18]:
```
plot_contour_and_paths(problem=rastrigin, paths=rastrigin_positions)
```



Question: How does the velocity norm evolve? Does it correlate with convergence speed?

Answer: Velocity norm evolves differently for different α values - for larger α values ($\alpha \in \{10^{-3}, 10^{-4}\}$) the velocity norm oscilates strongly, for smaller values ($\alpha \in \{10^{-5}, 10^{-6}\}$) the velocity norm is relatively low and changes smoothly. Velocity norm "correlates" with convergence speed, but not monotonically - high oscilations in the velocity norm cause less stable and slower convergence, while very low velocity norm leads to slower convergence (the best convergence is achieved for $\alpha = 10^{-5}$)

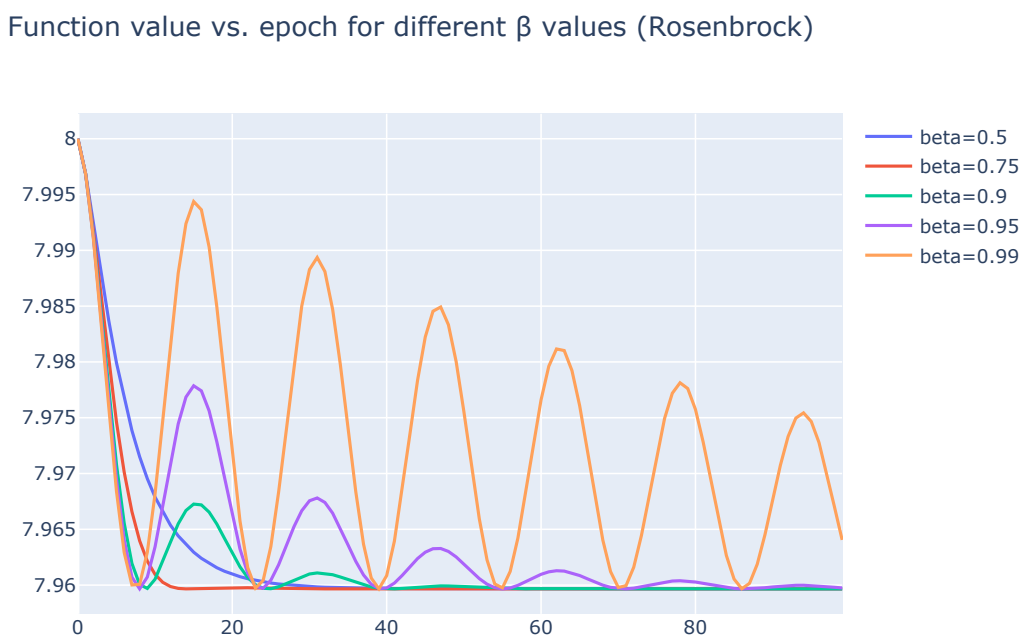
0.3 4. Momentum Hyperparameters

[19]:
```
alpha = 1e-4
initial_solution = np.array([2.0, 2.0])
number_of_epochs = 100
betas = [0.5, 0.75, 0.9, 0.95, 0.99]

rosenbrock_beta_function_values = []
```

```
rosenbrock_beta_velocity_norms = []
rosenbrock_beta_positions = []

rosenbrock = Rosenbrock()
```

[20]:
```
for beta in betas:
  _positions, _function_values, _velocity_norms = momentum(
    problem=rastrigin,
    initial_solution=initial_solution,
    alpha=alpha,
    beta=beta,
    number_of_epochs=number_of_epochs
  )

  rosenbrock_beta_function_values.append(_function_values)
  rosenbrock_beta_velocity_norms.append(_velocity_norms)
  rosenbrock_beta_positions.append(_positions)
```

[21]:
```
import plotly.graph_objects as go

fig = go.Figure()
for beta, function_values in zip(betas, rosenbrock_beta_function_values):
  fig.add_trace(go.Scatter(y=function_values, name=f"beta={beta}"))
fig.update_layout(
    title="Function value vs. epoch for different  values (Rosenbrock)"
)
```



Question: How does increasing β affect convergence? Stability?

Answer: For the `Rosenbrock` problem, increasing β slows the convergence and reduces stability (causing stronger oscillations). That might mean that for higher β values too much momentum is accumulated and the optimizer overshoots the optimum in the narrow valley. For $\beta = 0.99$ we do not even reach convergence.

0.4 5. Adam

[22]:
```
def adam(
    problem: Problem,
    initial_solution: np.ndarray,
    alpha: float,
    beta1: float,
    beta2: float,
    number_of_epochs: int,
    epsilon: float = 1e-8
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    dim = initial_solution.shape[0]
    v = np.zeros_like(initial_solution)
    m = np.zeros_like(initial_solution)
    w = initial_solution
```

```
    positions = np.ndarray(shape=(number_of_epochs, dim))
    function_values = np.ndarray(shape=(number_of_epochs))
    gradient_norms = np.ndarray(shape=(number_of_epochs))

    positions[0] = w
    function_values[0] = problem(w)
    gradient_norms[0] = np.linalg.norm(problem.grad(w))

    for i in range(1, number_of_epochs):
      m = beta1 * m + (1 - beta1) * problem.grad(w)
      v = beta2 * v + (1 - beta2) * problem.grad(w) ** 2
      m_hat = m / (1 - beta1 ** i)
      v_hat = v / (1 - beta2 ** i)

      w = w - alpha * m_hat / (np.sqrt(v_hat) + epsilon)

      positions[i] = w
      function_values[i] = problem(w)
      gradient_norms[i] = np.linalg.norm(problem.grad(w))

    return positions, function_values, gradient_norms
```

0.5 Comparing Adam with Momentum

[23]:
```
initial_solution = np.array([2.0, 2.0])
number_of_epochs = 100

momentum_alpha = 0.01
momentum_beta = 0.9

adam_alpha = 0.01
adam_beta1 = 0.9
adam_beta2 = 0.999
adam_epsilon = 1e-8
```

0.5.1 Sphere

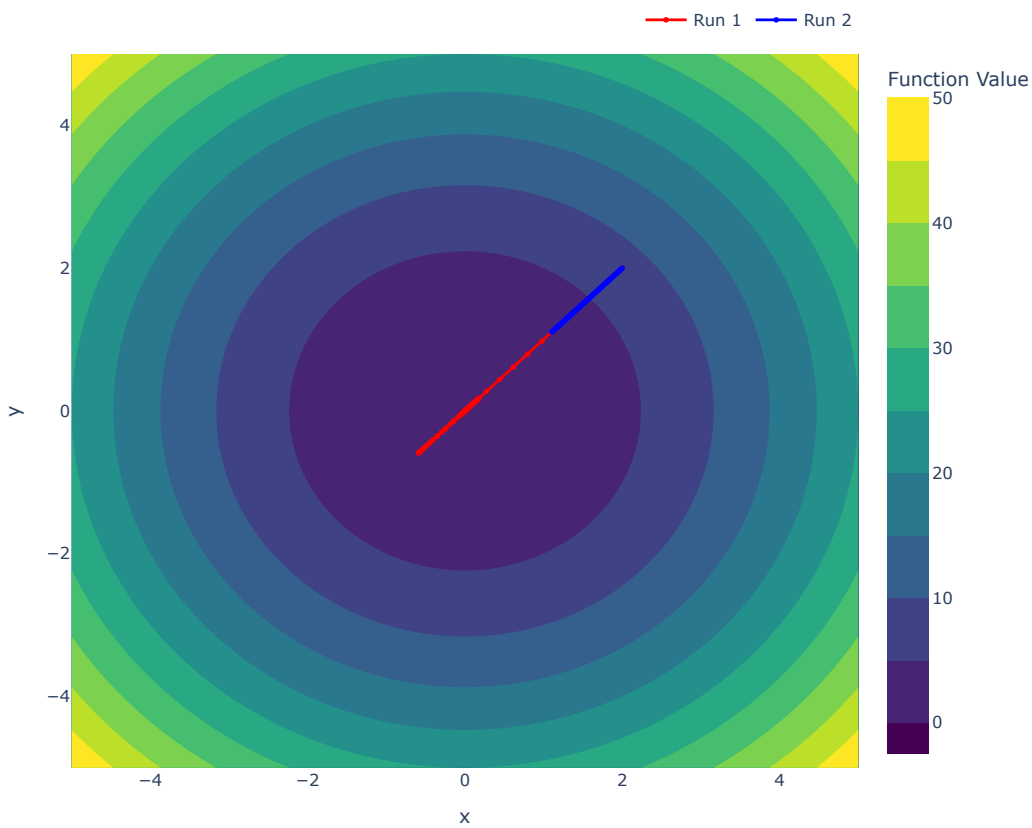
[24]:
```
sphere = Sphere()
```

[25]:
```
momentum_positions, momentum_function_values, momentum_velocity_norms =␣
 ↪momentum(
    problem=sphere,
    initial_solution=initial_solution,
    alpha=momentum_alpha,
    beta=momentum_beta,
    number_of_epochs=number_of_epochs
)
```

```
adam_positions, adam_function_values, adam_gradient_norms = adam(
    problem=sphere,
    initial_solution=initial_solution,
    alpha=adam_alpha,
    beta1=adam_beta1,
    beta2=adam_beta2,
    number_of_epochs=number_of_epochs,
    epsilon=adam_epsilon
)
```

[26]:
```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(y=momentum_function_values, name="Momentum"))
fig.add_trace(go.Scatter(y=adam_function_values, name="Adam"))
fig.update_layout(
    title="Function value vs. epoch (Sphere)"
)
```

[27]:
```
plot_contour_and_paths(problem=sphere, paths=[momentum_positions,␣
↪adam_positions])
```



0.5.2 Rosenbrock

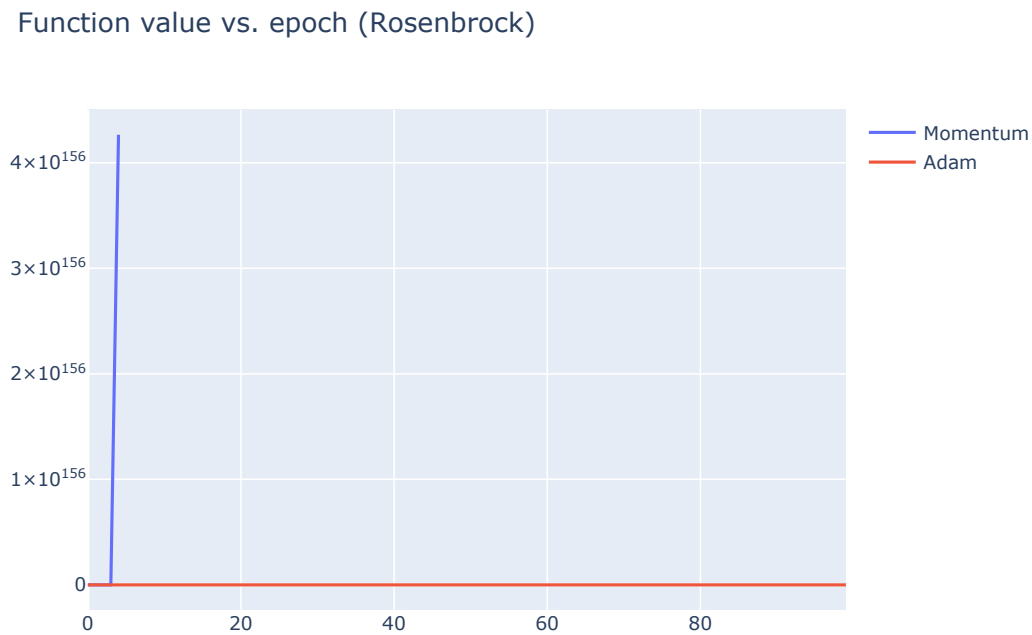
[28]:
```
rosenbrock = Rosenbrock()
```

[29]:
```
momentum_positions, momentum_function_values, momentum_velocity_norms =␣
↪momentum(
    problem=rosenbrock,
    initial_solution=initial_solution,
    alpha=momentum_alpha,
    beta=momentum_beta,
    number_of_epochs=number_of_epochs
)
```

```
adam_positions, adam_function_values, adam_gradient_norms = adam(
    problem=rosenbrock,
    initial_solution=initial_solution,
    alpha=adam_alpha,
    beta1=adam_beta1,
    beta2=adam_beta2,
    number_of_epochs=number_of_epochs,
    epsilon=adam_epsilon
)
```

```
/var/folders/90/55wz2vrd4375m33q50580xhc0000gn/T/ipykernel_99430/4269148045.py:2
4: RuntimeWarning: overflow encountered in square
  return np.sum(100.0 * (x[1:] - x[:-1]**2)**2 + (1 - x[:-1])**2)
/var/folders/90/55wz2vrd4375m33q50580xhc0000gn/T/ipykernel_99430/4269148045.py:2
9: RuntimeWarning: overflow encountered in scalar multiply
  grad[0] = -400 * x[0] * (x[1] - x[0]**2) - 2 * (1 - x[0])
/var/folders/90/55wz2vrd4375m33q50580xhc0000gn/T/ipykernel_99430/3410495211.py:2
1: RuntimeWarning: invalid value encountered in add
  v = beta * v + alpha * problem.grad(w)
```

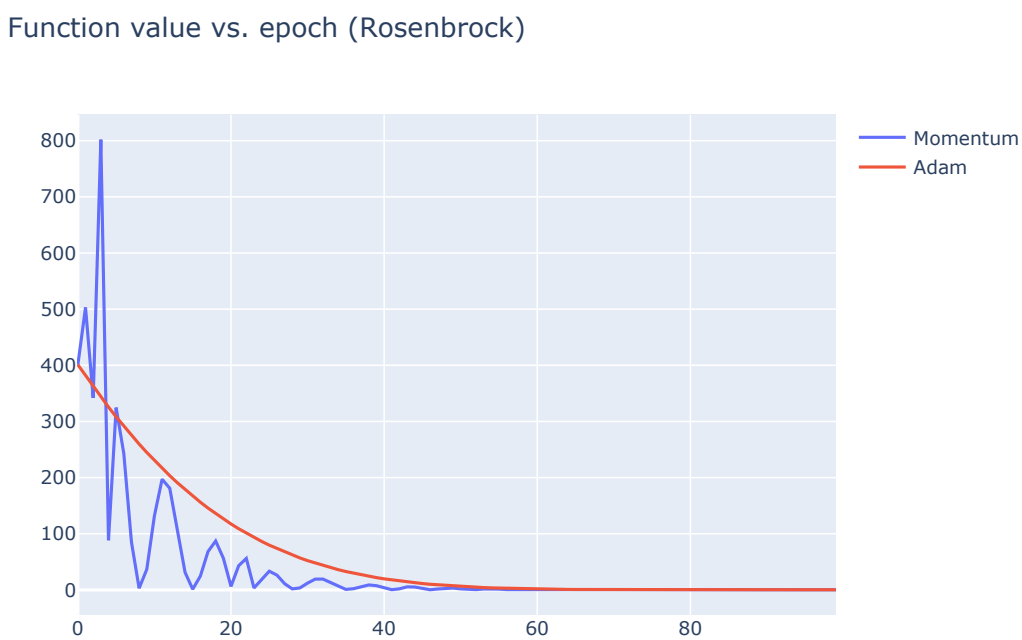
[30]:
```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(y=momentum_function_values, name="Momentum"))
fig.add_trace(go.Scatter(y=adam_function_values, name="Adam"))
fig.update_layout(
    title="Function value vs. epoch (Rosenbrock)"
)
```



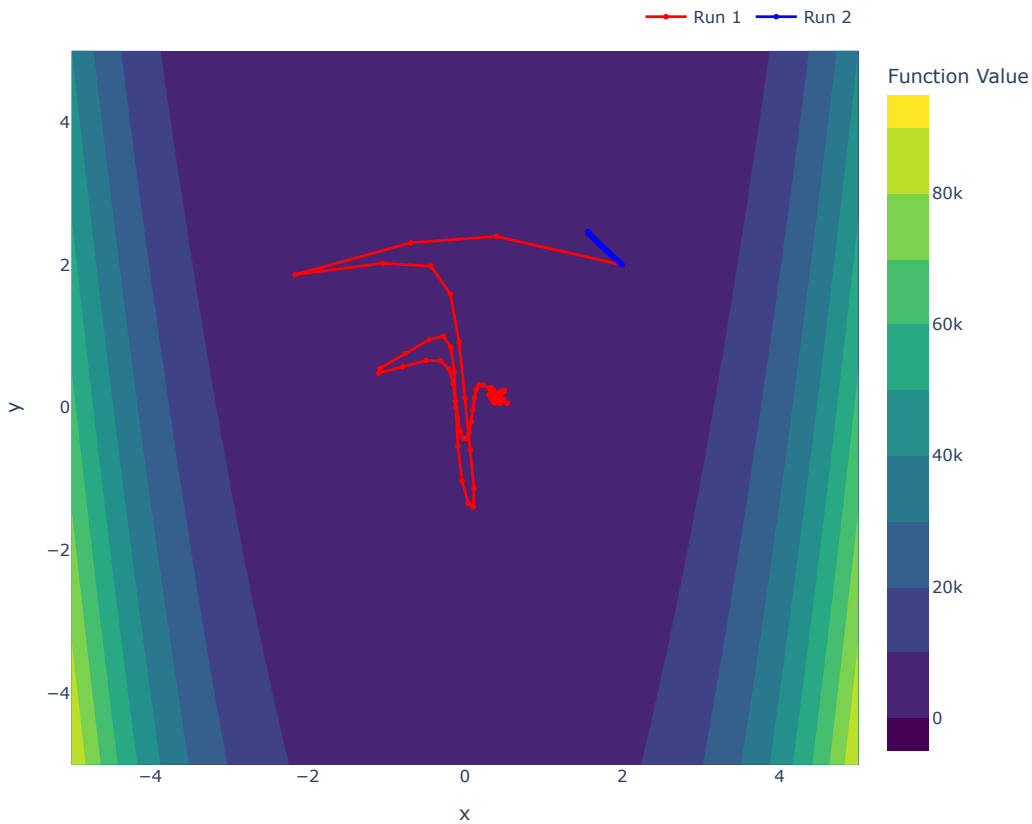
Note: for the `Rosenbrock` problem the `momentum` optimizer failed to converge due to gradient explosion (`Adam` converged beautifully). Because of that I decided to compare `momentum` and `Adam` one more time, this time with lower value for $\alpha_{momentum}$

[31]:
```
momentum_positions, momentum_function_values, momentum_velocity_norms =␣
↪momentum(
    problem=rosenbrock,
    initial_solution=initial_solution,
    alpha=0.001,
    beta=momentum_beta,
    number_of_epochs=number_of_epochs
)

adam_positions, adam_function_values, adam_gradient_norms = adam(
    problem=rosenbrock,
    initial_solution=initial_solution,
    alpha=adam_alpha,
    beta1=adam_beta1,
    beta2=adam_beta2,
    number_of_epochs=number_of_epochs,
    epsilon=adam_epsilon
)
```

[32]:
```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(y=momentum_function_values, name="Momentum"))
fig.add_trace(go.Scatter(y=adam_function_values, name="Adam"))
fig.update_layout(
    title="Function value vs. epoch (Rosenbrock)"
)
```



[33]:
```
plot_contour_and_paths(problem=rosenbrock, paths=[momentum_positions,␣
↪adam_positions])
```



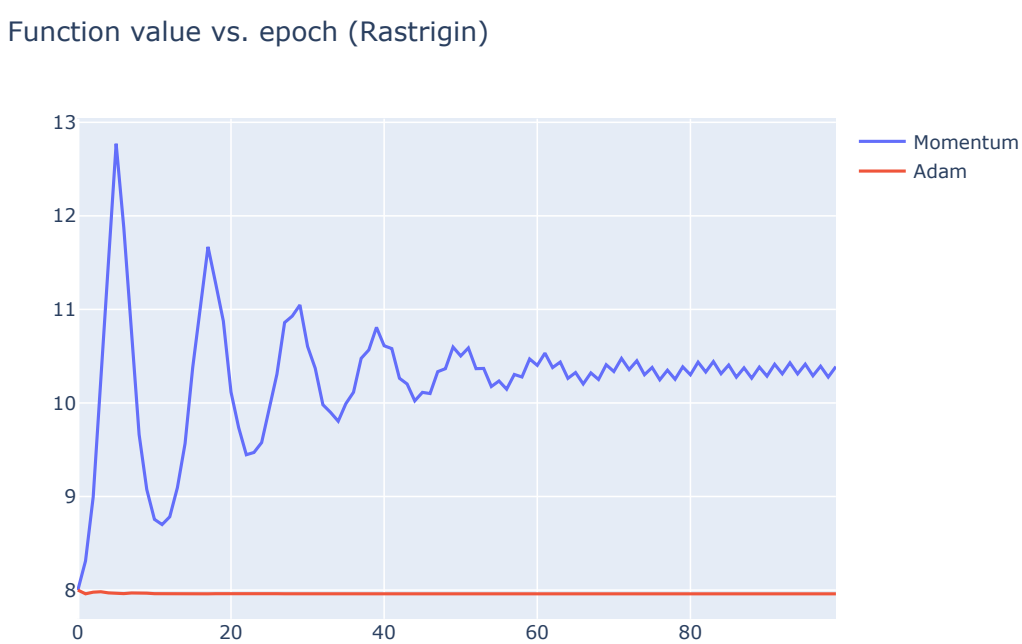
0.5.3 Rastrigin

[34]:
```
rastrigin = Rastrigin()
```

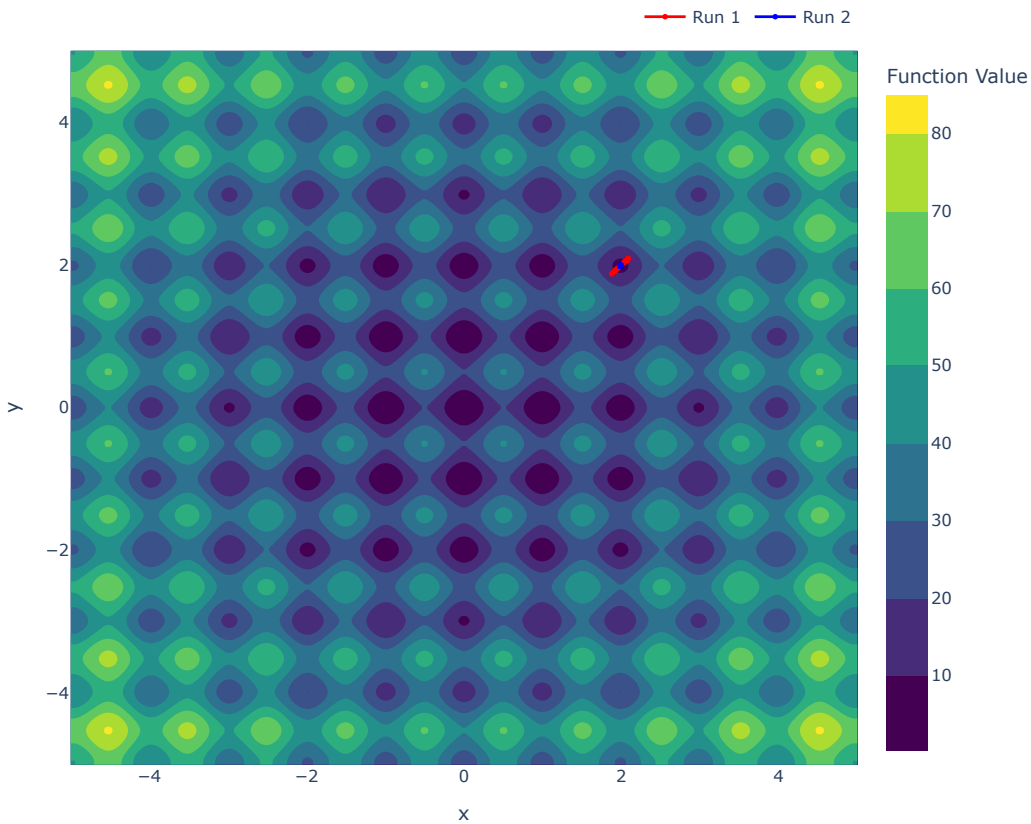
[35]:
```
momentum_positions, momentum_function_values, momentum_velocity_norms =␣
↪momentum(
    problem=rastrigin,
    initial_solution=initial_solution,
    alpha=momentum_alpha,
    beta=momentum_beta,
    number_of_epochs=number_of_epochs
)

adam_positions, adam_function_values, adam_gradient_norms = adam(
    problem=rastrigin,
    initial_solution=initial_solution,
```

```
    alpha=adam_alpha,
    beta1=adam_beta1,
    beta2=adam_beta2,
    number_of_epochs=number_of_epochs,
    epsilon=adam_epsilon
)
```

[36]:
```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(y=momentum_function_values, name="Momentum"))
fig.add_trace(go.Scatter(y=adam_function_values, name="Adam"))
fig.update_layout(
    title="Function value vs. epoch (Rastrigin)"
)
```



[37]:
```
plot_contour_and_paths(problem=rastrigin, paths=[momentum_positions,␣
↪adam_positions])
```


Run 1
Run 2
Function Value
80
70
60
50
40
30
20
10
4
2
0
−2
−4
y
−4
−2
0
2
4
x